

Évaluation des modèles, régularisation et cross-validation

Comprendre vraiment ses modèles avant de les déployer

Objectifs

- Choisir et interpréter les métriques adaptées à la régression
- Mettre en place une validation croisée robuste (K-Fold)
- Comprendre Ridge (L2) et Lasso (L1) et leurs effets sur les coefficients
- Comparer des modèles régularisés et sélectionner le meilleur

Prérequis (Cours 1 + TP 1)

- Régression linéaire simple et multiple
- Notion de MSE et de R^2 calculés sur les données d'entraînement
- Pourquoi un split train/test est obligatoire
- Utilisation de base de scikit-learn

1

Métriques d'évaluation en régression

Choisir le bon indicateur selon le contexte métier

1.1 Le zoo des métriques — quatre indicateurs essentiels

Avant de comparer des modèles, il faut choisir la bonne métrique. Chacune répond à une question différente et réagit différemment aux erreurs extrêmes.

Métrique	Formule simplifiée	Unité	Sensibilité outliers	Question posée
MSE	Moyenne des $(y - \hat{y})^2$	(unité y) ²	Très élevée	Quelle est l'erreur quadratique moyenne ?
RMSE	Racine carrée du MSE	Unité de y	Élevée	Quelle est l'erreur typique, dans l'unité de y ?
MAE	Moyenne des $ y - \hat{y} $	Unité de y	Modérée	Quelle est l'erreur absolue moyenne ?
R²	$1 - \text{MSE} / \text{Var}(y)$	Sans unité	Modérée	Quelle fraction de la variance est expliquée ?
R² ajusté	R ² pénalisé par p	Sans unité	Modérée	Idem, mais pénalisé si on ajoute des features inutiles

Règle empirique de choix

- **Si les grandes erreurs sont catastrophiques** (prédire la durée de vie d'un moteur d'avion, risque médical) → **MSE / RMSE** : la pénalisation quadratique force le modèle à éviter les grosses erreurs.
- **Si toutes les erreurs se valent** (prédire un délai de livraison à J+1 ou J+3 est équivalent) → **MAE** : robuste aux outliers, plus facile à interpréter.
- **Pour comparer des modèles de même complexité** → **R²** : indique la part de variance expliquée, indépendamment de l'échelle de y.
- **Pour comparer des modèles avec des nombres de features différents** → **R² ajusté** : pénalise l'ajout de features qui n'apportent rien.

1.2 Les trois pièges du R²

Le R² est la métrique la plus utilisée — et la plus souvent mal interprétée. Voici trois situations où il induit en erreur.

Piège 1 — Le R² peut être négatif

Le R² mesure si votre modèle est meilleur que de prédire simplement la moyenne de y. Si ce n'est pas le cas, le R² est négatif.

Définition du R²

$$R^2 = 1 - \text{MSE}(\text{modèle}) / \text{MSE}(\text{modèle nul})$$

où $\text{MSE}(\text{modèle nul}) = \text{Var}(y)$, c'est-à-dire la variance de y. Si votre modèle fait pire que prédire la moyenne partout, alors $\text{MSE}(\text{modèle}) > \text{Var}(y) \rightarrow R^2 < 0$.

⚠ Implication concrète d'un R² négatif

Un R² négatif signifie que votre modèle est moins précis qu'un modèle qui prédirait « le prix moyen » pour toutes les maisons.

C'est exactement ce qui se produit quand un modèle overfitté est déployé en production sur de nouvelles données.

Piège 2 — Le R^2 en entraînement augmente toujours quand on ajoute des features

En régression OLS classique, ajouter une feature — même du bruit pur sans aucun lien avec y — ne peut qu'augmenter ou maintenir le R^2 en entraînement. Il ne diminue jamais.

Cela signifie que le R^2 en entraînement n'est pas un critère valide pour choisir entre modèles de complexités différentes.

R^2 brut — le problème

- Augmente mécaniquement à chaque feature ajoutée
- Peut atteindre 1.0 en ajoutant suffisamment de variables bruit
- Donne une fausse impression que « plus de features = meilleur modèle »
- Ne pénalise pas la complexité inutile

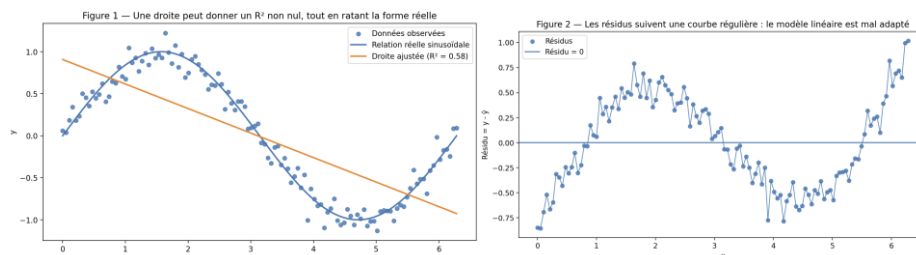
R^2 ajusté — la solution

- Intègre une pénalité pour le nombre de features p
- Peut diminuer si une feature n'apporte pas assez d'information
- Formule : $1 - (1 - R^2) \times (n-1) / (n-p-1)$
- Recommandé pour comparer des modèles avec des p différents

Piège 3 — Un R^2 élevé n'exclut pas des résidus structurés

Un R^2 de 0.70 ne signifie pas que le modèle est bien spécifié. Si les résidus montrent un pattern (courbe en U, variance croissante, structure périodique), cela révèle une relation que le modèle linéaire ne capture pas.

Exemple typique : modéliser une relation sinusoïdale avec une droite. Le R^2 peut être significatif, mais les résidus suivent une courbe régulière — preuve que le modèle est mal adapté à la forme des données.



→ Un R^2 "correct" ne suffit pas. Il faut aussi analyser les résidus pour vérifier si la forme du modèle est adaptée aux données.

1.3 MSE vs MAE — l'amplification quadratique

La différence fondamentale entre MSE et MAE est la façon dont elles traitent les grandes erreurs.

Erreur outlier	Impact sur MSE	Impact sur MAE
Erreur de 1	Contribution : $1^2 = 1$	Contribution : 1
Erreur de 5	Contribution : $5^2 = 25$	Contribution : 5
Erreur de 10	Contribution : $10^2 = 100$	Contribution : 10 (×4 vs erreur de 5)
Erreur de 30	Contribution : $30^2 = 900$	Contribution : 30 (×3 vs erreur de 10)

Exemple

Imaginons deux modèles :

Modèle A

Erreurs : 2, 2, 2, 2, 20

- $MAE = (2+2+2+2+20)/5=5,6$

- $MSE = (4+4+4+4+400)/5=83,2$

Modèle B

Erreurs : 5, 5, 5, 5, 5

- $MAE = 5$
- $MSE = 25$

Interprétation

- Avec MAE, les deux modèles semblent assez proches
- Avec MSE, le modèle A est beaucoup plus pénalisé à cause de la seule erreur de 20

Conclusion : MSE punit exponentiellement les grandes erreurs.

Choisir MSE revient à dire : je préfère plusieurs petites erreurs plutôt qu'une seule très grande
cela signifie :

- on accepte un modèle qui se trompe un peu partout
- mais on évite un modèle qui fait parfois une erreur énorme

C'est utile quand une grosse erreur est très coûteuse.

Exemples métiers

- prédire une dose médicale
- estimer un risque financier
- prévoir une demande critique en logistique

Dans ces cas, une très grosse erreur peut être grave, donc **MSE** est souvent préférable.

Quand préférer MAE ?

On préfère souvent **MAE** si :

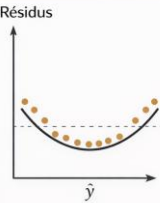
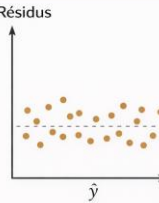
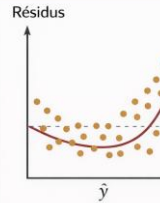
- on veut une mesure simple à interpréter
- on veut limiter l'effet des outliers
- on accepte qu'une grosse erreur ne "détruit" pas toute l'évaluation

Exemple :

- estimation de prix avec quelques biens atypiques
- prédiction de temps ou de ventes avec données bruitées

1.4 Analyse des résidus — les quatre diagnostics obligatoires

Un bon modèle ne se juge pas uniquement sur ses métriques globales. L'analyse des résidus révèle des problèmes invisibles dans le R^2 ou le RMSE.

Graphique	Ce qu'il révèle	Résultat attendu (modèle correct)	Problème si...
Résidus vs $\hat{y}$	Hétéroscédasticité, non-linéarité	Nuage horizontal sans structure	Structure visible, variance croissante
Résidus vs $\hat{y}$ Hétéroscédasticité, non-linéarité  Hétéroscédasticité, non-linéarité Nuage horizontal sans structure  Nuage horizontal sans structure Structure visible, variance croissante  Structure visible, variance croissante			

Scale-Location ($\sqrt{ \epsilon }$)	Varianse des erreurs selon $\hat{y}$	Tendance linéaire de pente ≈ 0	Pente positive = variance croissante
	Graphique « résidu vs valeurs prédites $\hat{y}$ » — trois situations typiques		
QQ-plot	Normalité de la distribution des résidus	Points alignés sur la diagonale théorique	Queues épaisses, points en S ou en courbe
	QQ-plot — trois situations typiques		
Histogramme résidus	Asymétrie, valeurs aberrantes, biais	Cloche symétrique centrée en 0	Asymétrie, pic décalé, queues épaisses
	Histogramme des résidus — trois situations typiques		

💡 Ces diagnostics sont-ils obligatoires ?

Pour la prédiction pure : non, un RMSE bas suffit si vous ne cherchez pas à interpréter.

Pour l'inférence (tester si β_1 est significativement différent de 0) : oui, les hypothèses de normalité et d'homoscédasticité sont nécessaires pour que les intervalles de confiance soient valides.

En pratique : toujours les produire. Ils révèlent souvent des transformations utiles (log y, log x) ou des outliers à traiter.

2

Validation croisée

Une estimation robuste de la performance

2.1 Le problème d'un seul split

Un split 80/20 avec un seul random_state donne une seule estimation de la performance. Cette estimation est aléatoire : elle dépend de quelles observations tombent dans le test.



Expérience de pensée

Imaginez que vous évaluez un modèle avec 50 splits différents (random_state = 0, 1, 2, ..., 49). Vous obtenez 50 valeurs de RMSE différentes, toutes valides mais toutes légèrement différentes.

L'écart entre la meilleure et la pire valeur peut représenter plusieurs pourcents de l'erreur.

Avec un seul split, vous ne savez pas si votre RMSE est le meilleur, le pire, ou la moyenne.

2.2 La solution : K-Fold Cross-Validation

La cross-validation K-Fold résout ce problème en répétant l'évaluation K fois, chaque fois avec un jeu de test différent. Le score final est la moyenne des K scores — bien plus stable et représentatif.

Fold 1	TEST	Train	Train	Train	Train
Fold 2	Train	TEST	Train	Train	Train
Fold 3	Train	Train	TEST	Train	Train
Fold 4	Train	Train	Train	TEST	Train
Fold 5	Train	Train	Train	Train	TEST

À chaque fold : on entraîne sur les K-1 parties colorées en blanc, on évalue sur la partie bleue (TEST). Le score final est la moyenne des K mesures $\pm$ l'écart-type (incertitude).

Choix de K : quel nombre de folds ?

K	Avantage	Inconvénient	Recommandé pour
5	Rapide, peu de calcul	Variance de l'estimation légèrement plus élevée	Grands datasets, prototypage rapide
10	Estimation plus stable	2× plus lent que K=5	Standard industrie — à préférer
n (LOOCV =Leave-One-Out Cross-Validation)	Variance minimale de l'estimation	Très lent, biais plus élevé sur petits datasets	Petits datasets (< 100 observations)

2.3 La règle d'or : le StandardScaler fait partie du fold

C'est l'erreur la plus subtile et la plus fréquente dans l'utilisation de la cross-validation.

✗ Mauvaise pratique (data leakage)

✓ Bonne pratique (Pipeline)

1. Standardiser tout X avec <code>fit_transform(X)</code> 2. Appliquer la cross-validation sur <code>X_standardisé</code> <i>Problème : le scaler a vu les données de test dans son fit() → les statistiques du test contaminent les folds d'entraînement.</i>	Créer un Pipeline : [<code>StandardScaler</code> → <code>LinearRegression</code>] Passer ce Pipeline directement à <code>cross_val_score()</code> <i>Le Pipeline garantit que le scaler est fitté uniquement sur les données d'entraînement de chaque fold.</i>
--	---

2.4 Lire les résultats de la cross-validation

La cross-validation retourne K scores, un par fold. Voici comment les interpréter.

Indicateur	Calcul	Interprétation
Moyenne des scores	Somme des K scores / K	Estimation robuste de la performance réelle
Écart-type	Dispersion des K scores autour de la moyenne	Incertitude : ± 0.01 = stable ; ± 0.05 = variable
Gap train – test	RMSE test moyen – RMSE train moyen	Mesure du biais : gap élevé = underfitting
Gap test – test	Max – min des K scores de test	Sensibilité aux données : si élevé, peu de données

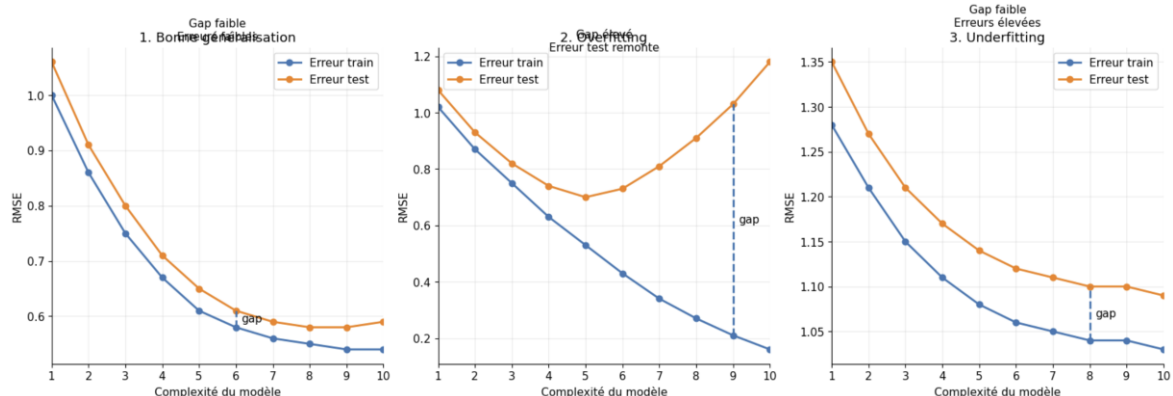
Comment interpréter le gap entre erreur train et erreur test ?

Gap proche de 0 : le modèle se comporte identiquement en train et en test. Bonne généralisation.

Gap élevé, erreur test $\gg$ erreur train : overfitting — le modèle a mémorisé le train.

Erreur train et erreur test élevées, gap faible : underfitting — le modèle n'est pas assez expressif.

→ Ce diagnostic guide directement la décision : augmenter la complexité (si underfitting) ou régulariser (si overfitting).



3

Régularisation Ridge et Lasso

Contrôler l'overfitting en ajoutant une contrainte sur les coefficients

3.1 Le problème : pourquoi les coefficients OLS deviennent instables

La régression OLS cherche les coefficients qui minimisent l'erreur sur les données d'entraînement — sans aucune autre contrainte. Dans deux situations, cela pose problème.

Situation 1 : trop de features (p proche de n) - Le modèle dispose de presque autant de paramètres que d'observations - Il peut interpoler parfaitement les données d'entraînement — y compris le bruit - Les coefficients prennent des valeurs très grandes, instables	Situation 2 : multicollinéarité forte - Deux features très corrélées (ex : AveRooms et AveBedrms dans California Housing) - La matrice $X^T X$ est quasi-singulière → les coefficients sont mathématiquement instables - Un tout petit changement dans les données peut faire varier β_1 de +100 à -80 → Les prédictions restent correctes mais l'interprétation des coefficients est invalide
--	--

3.2 L'idée de la régularisation

La régularisation consiste à ajouter une pénalité dans la fonction coût. Au lieu de minimiser uniquement l'erreur sur les données, on minimise un compromis entre erreur et taille des coefficients.

Fonction coût régularisée

Coût total = Erreur sur les données + $\lambda \times$ Pénalité(β)

λ (lambda) ou alpha dans sklearn : contrôle le poids de la pénalité. $\lambda=0 \rightarrow$ OLS classique. $\lambda \rightarrow \infty \rightarrow$ tous les $\beta \rightarrow 0$ (modèle nul). La valeur optimale se trouve par cross-validation.

OLS	Ridge	Lasso
Aucune contrainte	Pénalité L2 : $\Sigma \beta^2$	Pénalité L1 : $\Sigma \beta $
Tous les coefs libres Grande amplitude sur features bruit	Tous les coefs réduits vers 0 Aucun coef exactement nul	Certains coefs = 0 exactement Sélection automatique de variables

Oui — voici une explication **très simple** avec un exemple concret.

Idée générale

On veut prédire, par exemple, le **prix d'une maison** à partir de plusieurs variables :

- surface
- nombre de chambres
- âge du bien
- distance au centre
- jardin
- garage

- et aussi des variables moins utiles ou bruitées

Le modèle cherche une formule du type :

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Les **coefficients (β)** disent à quel point chaque variable influence la prédiction.

1) OLS

OLS = régression linéaire classique

Principe

Le modèle choisit les coefficients qui minimisent l'erreur, **sans aucune pénalité**.

Ce que cela implique

- tous les coefficients sont libres
- certains peuvent devenir très grands
- si certaines variables sont du bruit ou très corrélées entre elles, le modèle peut devenir instable

Exemple simple

On veut prédire le prix d'une maison avec :

- surface
- nombre de chambres
- distance au centre
- couleur de la boîte aux lettres

Supposons qu'après apprentissage, on obtienne :

- surface : **+3200**
- chambres : **+8000**
- distance : **-5000**
- boîte aux lettres : **+2700**

Le problème est que la dernière variable n'a probablement **pas de vraie utilité**, mais OLS peut quand même lui donner un poids non négligeable, surtout si les données sont bruitées.

Intuition

OLS essaie d'expliquer au maximum les données d'entraînement, parfois **en prenant trop au sérieux des variables peu utiles**.

2) Ridge

Ridge ajoute une **pénalité L2** :

$$\sum \beta_j^2$$

Donc il n'essaie pas seulement de minimiser l'erreur, il essaie aussi de **garder les coefficients petits**.

Effet

- tous les coefficients sont **réduits vers 0**
- mais **aucun n'est exactement nul** en général
- le modèle est plus stable
- très utile quand les variables sont nombreuses ou corrélées

Exemple simple

Avec les mêmes variables, Ridge pourrait donner :

- surface : **+2900**
- chambres : **+6500**
- distance : **-4200**
- boîte aux lettres : **+300**

On voit que :

- les coefficients utiles restent présents
- la variable peu utile est beaucoup moins influente
- mais elle n'est pas supprimée totalement

Intuition

Ridge dit :

“Je garde toutes les variables, mais je limite leur puissance.”

3) Lasso

Lasso ajoute une **pénalité L1** :

$$\sum |\beta_j|$$

Cette pénalité pousse certains coefficients à devenir **exactement 0**.

Effet

- certains coefficients sont conservés
- d'autres sont annulés
- cela fait une **sélection automatique de variables**

Exemple simple

Lasso pourrait donner :

- surface : **+3000**
- chambres : **+7000**
- distance : **-4500**
- boîte aux lettres : **0**

Ici, la variable *couleur de la boîte aux lettres* est carrément retirée du modèle.

Intuition

Lasso dit :

“Je garde seulement les variables vraiment utiles.”

Quand utiliser quoi ?

OLS

Quand :

- peu de variables
- peu de bruit
- relation simple et stable

Ridge

Quand :

- beaucoup de variables
- variables corrélées
- on veut stabiliser sans supprimer

Lasso

Quand :

- on veut un modèle plus simple
- on veut identifier les variables importantes
- certaines variables sont probablement inutiles

3.3 Ridge — régularisation L2

Principe : Ridge ajoute la somme des carrés des coefficients comme pénalité. Plus un coefficient est grand, plus il est pénalisé.

Fonction coût Ridge

$$\text{Coût Ridge} = \text{MSE} + \lambda \cdot \sum \beta_j^2$$

La pénalité est proportionnelle au carré de chaque coefficient. Tous les coefficients sont réduits vers zéro mais aucun n'atteint exactement zéro.

Ce que fait Ridge concrètement

- **Shrinkage uniforme** : tous les coefficients sont réduits proportionnellement à leur amplitude. Un coefficient de 10 est plus réduit qu'un coefficient de 0.5.
- **Aucune sélection de variables** : Ridge conserve toutes les features dans le modèle avec des coefficients non nuls. Si vous avez 50 features, Ridge en garde 50.
- **Stabilité face à la multicollinéarité** : quand deux features sont très corrélées, OLS rend leurs coefficients très instables. Ridge les répartit équitablement entre les deux features corrélées.
- **Alpha croissant = plus de régularisation** : à mesure qu'on augmente alpha, tous les coefficients se rapprochent de 0. Le chemin de régularisation montre cette convergence progressive.

Le chemin de régularisation Ridge

Le chemin de régularisation est un graphique qui montre comment chaque coefficient évolue en fonction d'alpha (de petit à grand). C'est l'outil de visualisation clé pour comprendre Ridge.

Lecture du chemin de régularisation Ridge

Alpha petit (gauche du graphique) : les coefficients ont les valeurs OLS — grands et potentiellement instables.

Alpha optimal (milieu) : les coefficients importants restent significatifs, les coefficients bruit sont proches de 0 mais non nuls.

Alpha grand (droite) : tous les coefficients convergent vers 0. Le modèle prédit simplement la moyenne de y.

3.4 Lasso — régularisation L1

Principe : Lasso remplace la pénalité quadratique par une pénalité sur la valeur absolue des coefficients. Cette différence mathématique apparemment mineure a une conséquence majeure : certains coefficients sont forcés à exactement zéro.

Fonction coût Lasso

$$\text{Coût Lasso} = \text{MSE} + \lambda \cdot \sum |\beta_j|$$

La pénalité est proportionnelle à la valeur absolue (et non au carré). La géométrie de cette contrainte crée des coins sur les axes → l'optimum tombe souvent sur un axe où $\beta_j = 0$.

Ce que fait Lasso concrètement

- **Sélection automatique de variables** : Lasso force certains coefficients à exactement zéro. Sur 50 features, Lasso peut n'en garder que 10 ou 15. C'est la propriété la plus précieuse de Lasso.
- **Modèle parcimonieux** : le modèle final est plus simple, plus interprétable, et potentiellement plus robuste aux nouvelles données.
- **Sensibilité à la multicollinéarité** : si deux features sont très corrélées, Lasso a tendance à en garder une et à annuler l'autre — le choix est quasi-aléatoire et peut changer selon les données.
- **Alpha croissant = plus de features annulées** : à mesure qu'on augmente alpha, de plus en plus de coefficients tombent à zéro. À alpha très grand, le modèle ne garde plus aucune feature.

Pourquoi L1 annule mais pas L2 ? — Intuition géométrique

La pénalité Ridge définit une contrainte en forme de boule (sphère). La pénalité Lasso définit une contrainte en forme de losange (diamant). La solution optimale du problème régularisé tend à se trouver aux intersections entre la surface d'erreur et la contrainte.

Les intersections avec la sphère (Ridge) sont sur une surface lisse — jamais sur les axes. Les intersections avec le losange (Lasso) sont souvent sur les coins — qui correspondent exactement aux axes, là où certains $\beta_j = 0$.

3.5 Comment choisir alpha — RidgeCV et LassoCV

En pratique, on ne choisit jamais alpha à la main. On utilise une cross-validation interne pour trouver automatiquement la valeur optimale.

Outil	Comment ça fonctionne	Avantage clé
RidgeCV	Utilise leave-one-out CV par défaut (exact et rapide pour Ridge grâce aux propriétés analytiques de L2)	Un seul appel à <code>fit()</code> suffit — pas de boucle manuelle sur les alphas
LassoCV	Utilise une K-Fold CV standard (nécessaire car Lasso n'a pas de solution analytique fermée)	Trouve l'alpha optimal ET ajuste le modèle en même temps — intégrable dans un Pipeline

3.6 Ridge vs Lasso vs ElasticNet — tableau de décision

Modèle	Pénalité	Utiliser quand...	Avantage clé	Limite principale
OLS	Aucune	$p \ll n$, pas de corrélations	Interprétabilité maximale	Instable si multicollinéarité ou $p \approx n$
Ridge	L2 : $\sum \beta^2$	Toutes les features contribuent un peu	Stabilité, gère la multicollinéarité	Ne sélectionne pas (tous coefs $\neq 0$)
Lasso	L1 : $\sum \beta $	Sélection de variables nécessaire, modèle clairsemé	Modèle parcimonieux et lisible	Instable si features très corrélées
ElasticNet	L1 + L2	Groupes de features corrélées	Sélection + stabilité	2 hyperparamètres à tuner

ElasticNet — quand l'utiliser ?

ElasticNet combine les pénalités L1 et L2 avec un paramètre de mélange p (`l1_ratio` dans `sklearn`). Quand $p = 1$, c'est Lasso. Quand $p = 0$, c'est Ridge.

Sa force principale : quand on a des groupes de features fortement corrélées et qu'on veut à la fois sélectionner des variables ET maintenir de la stabilité. Lasso seul garderait une feature arbitraire dans chaque groupe ; ElasticNet peut garder tout le groupe.

3.7 Sur quel dataset la régularisation est-elle utile ?

Un point qui surprend souvent : sur un dataset avec peu de features et beaucoup d'observations, Ridge et OLS donnent des résultats quasi-identiques. La régularisation n'est réellement utile que dans des situations précises.

 Quand la régularisation fait vraiment la différence

Quand p est grand par rapport à n (ratio $p/n > 0.1$) — c'est le cas du TP 2 (50 features, 500 observations).

Quand des features très corrélées rendent les coefficients OLS instables — multicollinéarité forte.

Quand on veut un modèle parcimonieux pour l'interprétation ou le déploiement — Lasso idéal.

Sur un grand dataset sain (California Housing : 20 640 obs., 8 features), OLS et Ridge sont quasi-équivalents.



Synthèse — Formules et workflow

Ce que vous devez maîtriser à l'issue de ce cours

Formules à retenir

Formule	Expression	À retenir
RMSE	$\sqrt{(1/n) \cdot \sum (y - \hat{y})^2}$	Même unité que y — à préférer à MSE pour la lisibilité
R ²	$1 - \text{MSE} / \text{Var}(y)$	$\in (-\infty, 1]$. R ² < 0 = pire que prédire la moyenne
R ² ajusté	$1 - (1 - R^2) \cdot (n-1) / (n-p-1)$	Pour comparer des modèles avec des p différents
CV score	$(1/K) \cdot \sum \text{score}_k \pm \text{std}$	Estimation robuste $\pm$ incertitude sur K folds
Coût Ridge	$\text{MSE} + \lambda \cdot \sum \beta_j^2$	Shrink tous les coefs, aucun à exactement zéro
Coût Lasso	$\text{MSE} + \lambda \cdot \sum \beta_j $	Force certains coefs à exactement zéro — sélection
ElasticNet	$\text{MSE} + \lambda \cdot (\rho \cdot \text{L1} + (1-\rho) \cdot \text{L2})$	$\rho=1 \rightarrow \text{Lasso}$ · $\rho=0 \rightarrow \text{Ridge}$ · $0 < \rho < 1 \rightarrow \text{mix}$

Workflow recommandé

Workflow complet — du split à l'interprétation

1. **Split train/test** — Toujours en premier, avant toute transformation
2. **Créer un Pipeline** — StandardScaler + RidgeCV ou LassoCV (alpha sélectionné automatiquement)
3. **Entraîner sur X_train** — Le Pipeline fit le scaler ET sélectionne alpha par CV interne
4. **Évaluer sur X_test** — Une seule fois — RMSE, R², analyse des résidus
5. **Valider avec cross_val_score** — 10-fold sur tout le dataset : RMSE moyen $\pm$ std
6. **Analyser les résidus** — 4 graphiques : résidus vs $\hat{y}$, scale-location, QQ-plot, histogramme
7. **Comparer les modèles** — Tableau OLS vs Ridge vs Lasso — RMSE test, R² test, nb features non-nulles